

MI DIVP API

1 Overview

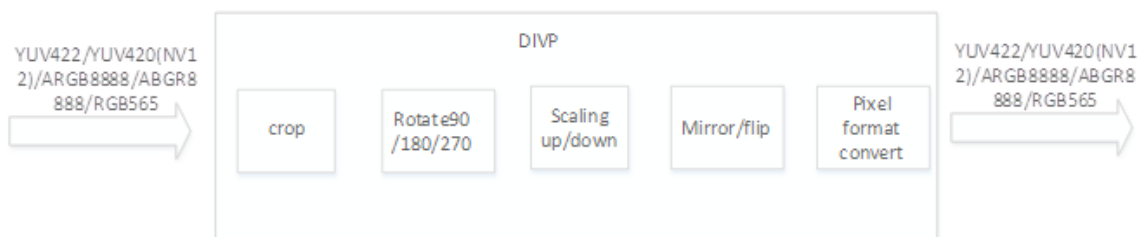
1.1. Module Description

DIVP supports preprocessing of an input image, such as cropping, etc., and then scaling each channel separately, and finally outputting images of different resolutions.

The specific image processing functions supported by DIVP include:

- cropping of images
- Image pixel format conversion
- image rotation
- Image mirror operation
- Stretching and scaling of images

1.2. Flow Diagram



Notice : Chips that do not support image rotation are listed in **Macaron** 、**Taiyaki** 、**Takoyaki** 、**Ispahan** series.

1.3. Keyword Description

keywords	illustrate
crop	image cropping
rotate	Image rotation
scaling up/down	Image stretching and scaling
Mirror/flip	Image mirror operation
Pixel format convert	Image pixel format conversion

2. API Reference

2.1. Function Module API

API name	Function
MI_DIVP_CreateChn [#22-mi_divp_createchn]	Create a DIVP channel
MI_DIVP_DestroyChn [#23-mi_divp_destroychn]	Destroy a DIVP channel
MI_DIVP_SetChnAttr [#24-mi_divp_setchnattr]	Set the properties of the DIVP channel
MI_DIVP_GetChnAttr [#25-mi_divp_getchnattr]	Get the properties of the DIVP channel
MI_DIVP_StartChn [#26-mi_divp_startchn]	open a channel
MI_DIVP_StopChn [#27-mi_divp_stopchn]	disable a channel
MI_DIVP_SetOutputPortAttr [#28-mi_divp_setoutputportattr]	Set the properties of the DIVP output port
MI_DIVP_GetOutputPortAttr [#29-mi_divp_getoutputportattr]	Get the properties of the DIVP output port
MI_DIVP_RefreshChn [#210-mi_divp_refreshchn]	Refresh a channel in a paused state
MI_DIVP_StretchBuf [#211-mi_divp_stretchbuf]	Scale, stretch, and crop image data in specified memory
MI_DIVP_InitDev [#212-mi_divp_initdev]	Initialize DIVP device
MI_DIVP_DeInitDev [#213-mi_divp_deinitdev]	Deinitialize DIVP device
MI_DIVP_StretchBufEx [#214-mi_divp_stretchbufex]	Scale, stretch, crop the image data in the specified memory area, and output to the target memory

2.2. MI_DIVP_CreateChn

- Function
Create a new DIVP channel.
- grammar

```
MI_S32 MI_DIVP_CreateChn (
    MI_DIVP_CHN DivpChn ,
    MI_DIVP_ChnAttr_t * pstAttr );
```

- formal parameter

parameter name	describe	input Output
DivpChn	Returns the ID of the created DIVP channel, DIVP supports up to 6 channels, the value range is [0,5]	enter
pstAttr	Pointer to the DVIP channel properties, used to set the properties of the channel being created	enter

- return value
 - MI_SUCCESS successfully created a new DIVP channel.
 - MI_DIVP_ERR_FAILED Failed to create DIVP channel
 - MI_DIVP_ERR_NO_RESOUCE Insufficient system resources, failed to create DIVP channel
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice

The DIVP module supports up to 6 channels. When the total number of channels created by the upper layer is greater than 6 or the system resources are insufficient, the creation will fail.

- Example

```
MI_DIVP_CHN  u32ChnId  =  0 ;

MI_DIVP_ChnAttr_t  stDivpChnAttr ;

MI_DIVP_OutputPortAttr_t  stDivpOutputPortAttr ;

memset ( & stDivpChnAttr , 0 , sizeof ( MI_DIVP_ChnAttr_t ));

memset ( & stDivpOutputPortAttr , 0 , sizeof (
MI_DIVP_OutputPortAttr_t ));

stDivpChnAttr . bHorMirror  =  false ;

stDivpChnAttr . bVerMirror  =  false ;

stDivpChnAttr . eDiType  =  E_MI_DIVP_DI_TYPE_OFF ;

stDivpChnAttr . eRotateType  =  E_MI_SYS_ROTATE_90 ;

stDivpChnAttr . eTnrLevel  =  E_MI_DIVP_TNR_LEVEL_OFF ;

stDivpChnAttr . stCropRect . u16X  =  0 ;

stDivpChnAttr . stCropRect . u16Y  =  0 ;

stDivpChnAttr . stCropRect . u16Width  =  1280 ;

stDivpChnAttr . stCropRect . u16Height  =  720 ;

stDivpChnAttr . u32MaxWidth  =  1920 ;

stDivpChnAttr.u32MaxHeight = 1080 ; _ _

MI_DIVP_CreateChn ( u32ChnId , & stDivpChnAttr );

MI_DIVP_GetChnAttr ( u32ChnId , & stDivpChnAttr );

stDivpChnAttr . stCropRect . u16X  =  0 ;

stDivpChnAttr . stCropRect . u16Y  =  0 ;
```

```

stDivpChnAttr . stCropRect . u16Width  = 1920 ;

stDivpChnAttr . stCropRect . u16Height = 1080 ;

MI_DIVP_SetChnAttr ( u32ChnId , & stDivpChnAttr );

stDivpOutputPortAttr . eCompMode =
E_MI_SYS_COMPRESS_MODE_NONE ;

stDivpOutputPortAttr . ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV422_YUYV ;

stDivpOutputPortAttr . u32Width  = 1920 ;

stDivpOutputPortAttr.u32Height = 1080 ; _ _

MI_DIVP_SetOutputPortAttr ( u32ChnId , & stDivpOutputPortAttr
);

MI_DIVP_StartChn ( u32ChnId );

// exit flow

MI_DIVP_StopChn ( u32ChnId );

MI_DIVP_DestroyChn ( u32ChnId );

```

- related topic

[MI_DIVP_DestroyChn](#) [#23-mi_divp_destroychn]

2.3. MI_DIVP_DestroyChn

- Function

Destroy a DIVP channel.

- grammar

```
MI_S32 MI_DIVP_DestroyChn(MI_DIVP_CHN DivpChn);
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the destroyed DVIP channel	enter

- return value
 - MI_SUCCESS successfully destroyed the DVIP channel.
 - MI_DIVP_ERR_FAILED Failed to destroy DVIP channel.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- related topic

[MI_DIVP_CreateChn](#) [#22-mi_divp_createchn]

2.4. MI_DIVP_SetChnAttr

- Function

Set the properties of the DIVP channel.
- grammar

```
MI_S32 MI_DIVP_SetChnAttr (
    MI_DIVP_CHN DivpChn ,
    MI_DIVP_ChnAttr_t * pstAttr );
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter
pstAttr	Structure pointer to the properties of the set DVIP channel	enter

- return value
 - MI_SUCCESS successfully set the properties of the DIVP channel
 - MI_DIVP_ERR_FAILED Failed to set the attributes of DIVP channel, there is a problem with the parameters passed in by MI_ERR_INVALID_PARAMETER, please refer to [MI_DIVP_ChnAttr_t](#) [#mi_divp_chnattr_t]
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice

The maximum width and maximum height of the input image in the channel property structure members are static properties and cannot be changed after the channel is created.
- related topic

[MI_DIVP_GetChnAttr](#) [#25-mi_divp_getchnattr]

2.5. MI_DIVP_GetChnAttr

- Function

Get the properties of the DIVP channel.
- grammar

```
MI_S32 MI_DIVP_GetChnAttr (
```

```
MI_DIVP_CHN DivpChn ,  
  
MI_DIVP_ChnAttr_t * pstAttr );
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter
pstAttr	Structure pointer to the properties of the returned DVIP channel	output

- return value
 - MI_SUCCESS successfully obtained the attributes of the DIVP channel.
 - MI_DIVP_ERR_FAILED Failed to get properties of DIVP channel.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice

Returns the initial value at creation when it is first fetched.
- related topic

[MI_DIVP_SetChnAttr](#) [#24-mi_divp_setchnattr]

2.6. MI_DIVP_StartChn

- Function

Open a channel.
- grammar

```
MI_S32 MI_DIVP_StartChn(MI_DIVP_CHN DivpChn);
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter

- return value

- MI_SUCCESS successfully enabled DIVP channel.
- MI_DIVP_ERR_FAILED Failed to enable DIVP channel.

- rely

- Header files: mi_divp.h, mi_divp_datatype.h
- Library file: libmi_divp.so

- Notice

The channel must have been created and not destroyed.

- related topic

[MI_DIVP_StopChn](#) [#27-mi_divp_stopchn]

2.7. MI_DIVP_StopChn

- Function

Disable a channel.

- grammar

```
MI_S32 MI_DIVP_StopChn(MI_DIVP_CHN DivpChn);
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter

- return value
 - MI_SUCCESS successfully disabled the DIVP channel.
 - MI_DIVP_ERR_FAILED Failed to disable DIVP channel.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice

Repeatedly disabling a channel returns MI_SUCCESS.
- related topic

[MI_DIVP_StartChn](#) [#26-mi_divp_startchn]

2.8. MI_DIVP_SetOutputPortAttr

- Function

Properties of the output port on the DIVP channel.
- grammar

```
MI_S32 MI_DIVP_SetOutputPortAttr (
    MI_DIVP_CHN DivpChn ,
    MI_DIVP_OutputPortAttr_t * pstOutputPortAttr );
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter
pstOutputPortAttr	A pointer to the property of the Output port	enter

- return value

- MI_SUCCESS successfully set the attribute of the output port bound by the DIVP channel.
- MI_DIVP_ERR_FAILED Failed to set the properties of the output port of the DIVP channel binding.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- related topic
 - [MI_DIVP_GetOutputPortAttr](#) [#29-mi_divp_getoutputportattr]

2.9. MI_DIVP_GetOutputPortAttr

- Function

Get the properties of the DIVP output port.
- grammar

```
MI_S32 MI_DIVP_GetOutputPortAttr (
    MI_DIVP_CHN DivpChn ,
    MI_DIVP_OutputPortAttr_t * pstOutputPortAttr );
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the DIVP channel	enter
pstOutputPortAttr	A pointer to the property of the Output port	output

- return value
 - MI_SUCCESS successfully obtained the properties of the output port bound to the DIVP channel.

- MI_DIVP_ERR_FAILED Failed to get the properties of the output port bound to the DIVP channel.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- related topic
 - [MI_DIVP_SetOutputPortAttr](#) [#28-mi_divp_setoutputportattr]

2.10. MI_DIVP_RefreshChn

- Function

Refresh the DIVP channel in the paused state.
- grammar

```
MI_S32 MI_DIVP_RefreshChn (  
  
MI_DIVP_CHN DivpChn );
```

- formal parameter

parameter name	describe	input Output
DivpChn	ID of the channel being flushed	enter

- return value
 - MI_SUCCESS successfully refreshed the DIVP channel.
 - MI_DIVP_ERR_NO_CONTENT This channel has no content.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice

This interface is suitable for channel refresh in a paused state.

- Example

In the scenario where DIVP is bound to VDEC, calling this interface, DIVP can keep the last frame after VDEC stops streaming, and repeat the processing and then send it to the backend. DIVP will not process new pictures until VDEC restarts streaming.

2.11. MI_DIVP_StretchBuf

- Function

Scale, stretch, and crop the image data in the specified memory area, and output to the target memory.

- grammar

```
MI_S32 MI_DIVP_StretchBuf(MI_DIVP_DirectBuf_t *pstSrcBuf,
MI_SYS_WindowRect_t *pstSrcCrop, MI_DIVP_DirectBuf_t
*pstDstBuf);
```

2.12. MI_DIVP_InitDev

- describe

Initialize the divp device.

- grammar

```
MI_S32 MI_DIVP_InitDev(MI_DIVP_InitParam_t *pstInitParam);
```

- parameter

parameter name	describe	input Output
pstInitParam	Device initialization parameters	enter

- return value
 - MI_OK success
 - Non-MI_OK failure, refer to the [error code](#) [#jump] .
 - rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
-

2.13. MI_DIVP_DeInitDev

- describe
Anti-initial divp device.
- grammar

```
MI_S32 MI_DIVP_DeInitDev(void);
```

- return value
 - MI_OK is successful.
 - Non-MI_OK failure, refer to the [error code](#) [#jump] .
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so
- Notice
 - This function must be called after initializing the device, otherwise it returns failure.
 - If this interface is not called before the app exits, the device will be automatically deinitialized internally.
- formal parameter

Parameter Name	Description	Input/Output	--- --- ---	Parameter Name	Description	Input/Output	--- --- ---
pstSrcBuf	The structure pointer used to store the memory information of the image to be processed, cannot be NULL	Input		pstSrcCrop	The structure pointer of		

image cropping related information, if you don't need to do cropping, you can pass NULL| input| |pstDstBuf| The structure pointer to save the image memory information after processing, cannot be NULL| input|

- return value
 - MI_SUCCESS Image processing was successful.
 - MI_DIVP_ERR_FAILED Image processing failed.
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so

- Notice

This interface only supports the pixel format of YUV420SP or ARGB8888.

- Example

```
#define SRC_WIDTH 1280 #define SRC_HEIGHT 720 #define
SRC_BUFF_STRIDE ( ALIGN_UP ( SRC_WIDTH , 16 ) ) #define
SRC_BUFF_SIZE ( SRC_BUFF_STRIDE * SRC_HEIGHT * 3 / 2 ) _ _ _

#define DST_WIDTH 640 #define DST_HEIGHT 480 #define
DST_BUFF_STRIDE ( ALIGN_UP ( DST_WIDTH , 16 ) ) #define
DST_BUFF_SIZE ( DST_BUFF_STRIDE * DST_HEIGHT * 3 / 2 ) _ _ _

#define CROP_X 200 #define CROP_Y 100 #define CROP_W 68
#define CROP_H 48 _ _ _ _

int main ( void )
{
    MI_PHY phySrcBufAddr = 0 ;
    MI_PHY phyDstBufAddr = 0 ;
    MI_DIVP_DirectBuf_t stSrcBuf ;
    MI_DIVP_DirectBuf_t stDstBuf ;
    MI_SYS_WindowRect_t stSrcCrop ;
```

```

MI_SYS_MMA_Alloc(NULL, SRC_BUFF_SIZE, &phySrcBufAddr) ;
MI_SYS_MMA_Alloc(NULL, DST_BUFF_SIZE, &phyDstBufAddr) ;

stSrcBuf.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420 ;
stSrcBuf.u32Width = SRC_WIDTH ;
stSrcBuf.u32Height = SRC_HEIGHT ;
stSrcBuf.u32Stride [ 0 ] = SRC_BUFF_STRIDE ;
stSrcBuf.u32Stride [ 1 ] = SRC_BUFF_STRIDE ;
stSrcBuf.phyAddr [ 0 ] = phySrcBufAddr ;
stSrcBuf.phyAddr [ 1 ] = stSrcBuf.phyAddr [ 0 ] +
SRC_BUFF_STRIDE*SRC_HEIGHT ;
stDstBuf.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420 ;
stDstBuf.u32Width = DST_WIDTH ;
stDstBuf.u32Height = DST_HEIGHT ;
stDstBuf.u32Stride [ 0 ] = DST_BUFF_STRIDE ;
stDstBuf.u32Stride [ 1 ] = DST_BUFF_STRIDE ;
stDstBuf.phyAddr [ 0 ] = phyDstBufAddr ;
stDstBuf.phyAddr [ 1 ] = stDstBuf.phyAddr [ 0 ] +
DST_BUFF_STRIDE*DST_HEIGHT ;

stSrcCrop.u16X = CROP_X ;
stSrcCrop.u16Y = CROP_Y ;
stSrcCrop.u16Width = CROP_W ;
stSrcCrop.u16Height = CROP_H ;

if(FillSrcBuf("./1280x720_yuv420.yuv", &stSrcBuf))
return NULL ;
if(MI_SUCCESS == MI_DIVP_StretchBuf(&stSrcBuf,
&stSrcCrop, &stDstBuf))
{
    if(DumpDstBuf(&stDstBuf))
        return NULL ;
}

MI_SYS_MMA_Free ( phySrcBufAddr );
MI_SYS_MMA_Free ( phyDstBufAddr );
}

// fill src buff with yuv420 / argb8888 image data
static int FillSrcBuf ( const char * FilePath ,
MI_DIVP_DirectBuf_t * pstDirectSrcBuf )
{
    int ret = 0 ;
    FILE *fp ;
    void *pVirSrcBufAddr = NULL ;
    int LineIdx = 0 ;

```

```

int  ReadSize  =  0 ;

fp  =  fopen(FilePath,"r") ;
if(!fp)
{
    divp_ut_dbg("open  file [ % s ]  failed\n",FilePath) ;
    ret  =  -1 ;
    goto  EXIT ;
}

MI_SYS_Mmap ( pstDirectSrcBuf-> phyAddr [ 0 ] ,
SRC_BUFF_SIZE , & pVirSrcBufAddr , FALSE );
if (! pVirSrcBufAddr )
{
    divp_ut_dbg("mmap  dst  buff  failed\n") ;
    ret  =  -1 ;
    goto  EXIT ;
}

for ( LineIdx  =  0 ; LineIdx  <  SRC_HEIGHT * 3 / 2 ;
LineIdx ++ )
{
    ReadSize  +=
fread(pVirSrcBufAddr+LineIdx*SRC_BUFF_STRIDE,  1,  SRC_WIDTH,
fp) ;
}
if ( ReadSize  <  SRC_WIDTH * SRC_HEIGHT * 3 / 2 )
{
    fseek(fp,  0,  SEEK_SET) ;
    ReadSize  =  0 ;
    for(LineIdx  =  0 ; LineIdx  <  SRC_HEIGHT*3/2 ;
LineIdx++)
    {
        ReadSize  +=
fread(pVirSrcBufAddr+LineIdx*SRC_BUFF_STRIDE,  1,  SRC_WIDTH,
fp) ;
    }
    if ( ReadSize  <  SRC_WIDTH * SRC_HEIGHT * 3 / 2 )
    {
        divp_ut_dbg("read  file  failed,  read  size : % d
\n ",ReadSize);
        ret  =  -1;
        goto  EXIT;
    }
}

EXIT:
if(fp)
    fclose(fp);

```

```

        if(pVirSrcBufAddr)
            MI_SYS_Munmap(pVirSrcBufAddr, SRC_BUFF_SIZE);

        return ret;
    }

    //image processing result is stored in dst buff
    static int DumpDstBuf(MI_DIVP_DirectBuf_t *pstDirectDstBuf)
    {
        int ret = 0;
        FILE *fp;
        void *pVirDstBufAddr = NULL;
        int LineIdx = 0;
        int WriteSize = 0;
        char outputfile [ 128 ] ;
        struct timeval timestamp;

        gettimeofday(&timestamp, 0);
        sprintf(outputfile, " output_ % dx % d_ % d_ % 08 d . yuv
",pstDirectDstBuf->u32Width,pstDirectDstBuf->u32Height,
(int)timestamp.tv_sec,(int)timestamp.tv_usec) ;

        fp = fopen(outputfile," w + ");
        if(!fp)
        {
            divp_ut_dbg(" open file [ % s ] failed \ n
",outputfile);
            ret = -1;
            goto EXIT;
        }

        MI_SYS_Mmap(pstDirectDstBuf->phyAddr [ 0 ] , DST_BUFF_SIZE,
&pVirDstBufAddr, FALSE);
        if(!pVirDstBufAddr)
        {
            divp_ut_dbg(" mmap dst buff failed \ n ");
            ret = -1;
            goto EXIT;
        }

        for(LineIdx = 0; LineIdx < DST_HEIGHT*3/2; LineIdx++)
        {
            WriteSize +=
fwrite(pVirDstBufAddr+LineIdx*DST_BUFF_STRIDE, 1, DST_WIDTH,
fp);
        }
        if(WriteSize < DST_WIDTH*DST_HEIGHT*3/2)
        {
            divp_ut_dbg (" write file failed , write size :% d

```

```
\ n ",WriteSize);
    }
    fflush(fp);
    sync();
    divp_ut_dbg(" save stretch dst buff to [ % s ] \ n " ,
outputfile );

EXIT :
    if ( fp )
        fclose ( fp );
    if(pVirDstBufAddr)
        MI_SYS_Munmap(pVirDstBufAddr,  DST_BUFF_SIZE) ;

    return 0 ;
}
```

2.14. MI_DIVP_StretchBufEx

- describe

Scale, stretch, and crop the image data in the specified memory area, and output to the target memory.

- grammar

```
MI_S32 MI_DIVP_StretchBufEx(MI_DIVP_DirectBuf_t *pstSrcBuf,
MI_SYS_WindowRect_t *pstSrcCrop, MI_DIVP_DirectBuf_t
*pstDstBuf, MI_DIVP_FilterType_e eFilterType);
```

- parameter

parameter name	describe	input Output
pstSrcBuf	Source buffer information pointer	enter
pstSrcCrop	Crop Information Pointer	enter
pstDstBuf	Target buffer information pointer	enter
eFilterType	The filter type for scaling.	enter

- return value
 - MI_OK success
 - Non-MI_OK failure, refer to the [error code](#) [jump] .
- rely
 - Header files: mi_divp.h, mi_divp_datatype.h
 - Library file: libmi_divp.so

3. DIVP data type

3.1. Data Type Definition

type of data	definition
MI_DIVP_DiType_e [#32-mi_divp_ditype_e]	Define the type of deinterlace of DIVP
MI_DIVP_TnrLevel_e [#33-mi_divp_tnrlevel_e]	Defining the level of DIVP TNR
MI_DIVP_FilterType_e [#34-mi-divp_filtertype_e]	Define DIVP scaling filter type
MI_DIVP_OutputPortAttr_t [#35-mi_divp_outputportattr_t]	Define the attribute parameters of the output port of the DIVP binding
MI_DIVP_ChnAttr_t [#36-mi_divp_chnattr_t]	Attribute parameters that define the DIVP channel
MI_DIVP_DirectBuf_t [#37-mi_divp_directbuf_t]	Memory information structure for image processing
MI_DIVP_InitParam_t [#38-mi_divp_initparam_t]	Define DIVP device initialization parameters

Note: This section has covered all important data types, some data types not listed, please refer to `mi_divp_datatype.h`

3.2. MI_DIVP_DiType_e

- illustrate
Defines the type of deinterlace for DIVP.
- definition

```
typedef enum
{
    E_MI_DIVP_DITYPE_OFF , // off

    E_MI_DIVP_DITYPE_2D , /// 2.5D DI _ _ _

    E_MI_DIVP_DITYPE_3D , /// 3D DI _

    E_MI_DIVP_DITYPE_NUM ,
} MI_DIVP_DiType_e ;
```

- member

member	describe
E_MI_DIVP_DITYPE_OFF	Close deinterlace on DIVP channel
E_MI_DIVP_DITYPE_2D	Enable 2.5D deinterlace on DIVP channel
E_MI_DIVP_DITYPE_3D	Enable 3D deinterlace on DIVP channel
E_MI_DIVP_DITYPE_NUM	Number of deinterlace types on the DIVP channel

- Precautions
 - TNR must be enabled when DI is enabled. MSR930 can only support 3D DI. The TNR level is set to `E_MI_DIVP_TNRLEVEL_MIDDLE`.

- 3D DI conflicts with rotation, and the two functions cannot be turned on at the same time.
 - The chips that do not support the DI function are as follows:
 - Pretzel
 - Macaron
 - Taiyaki/Takoyaki
 - Pudding
 - Ispahan
 - Ikayaki
-

3.3. MI_DIVP_TnrLevel_e

- illustrate

Defines the level of DIVP TNR.

- definition

```
typedef enum
{
    E_MI_DIVP_TNRLEVEL_OFF ,
    E_MI_DIVP_TNRLEVEL_LOW ,
    E_MI_DIVP_TNRLEVEL_MIDDLE ,
    E_MI_DIVP_TNRLEVEL_HIGH ,
    E_MI_DIVP_TNRLEVEL_NUM ,
} MI_DIVP_TnrLevel_e ;
```

- member
-

member	describe
E_MI_DIVP_TNRLEVEL_OFF	Turn off TNR on DIVP channel
E_MI_DIVP_TNRLEVEL_LOW	Enable weak level TNR on DIVP channel
E_MI_DIVP_TNRLEVEL_MIDDLE	Enable medium level TNR on DIVP channel
E_MI_DIVP_TNRLEVEL_HIGH	Enable strong TNR on DIVP channel
E_MI_DIVP_TNRLEVEL_NUM	Number of TNR levels on DIVP channel

- Precautions
 - The MSR930 supports TNR, but does not support adjusting the TNR level.
 - The chips that do not support the TNR function are as follows:
 - Pretzel
 - Macaron
 - Taiyaki/Takoyaki
 - Pudding
 - Ispahan
 - Ikayaki

3.4. MI_DIVP_FilterType_e

- illustrate

Define the DIVP scaling filter type.
- definition

```
typedef enum
{
    E_MI_DIVP_FILTER_TYPE_AUTO,
    E_MI_DIVP_FILTER_TYPE_BYPASS,
```

```
E_MI_DIVP_FILTER_TYPE_BILINEAR,  
E_MI_DIVP_FILTER_TYPE_FILTER,  
} MI_DIVP_FilterType_e;
```

- member

member	describe
E_MI_DIVP_FILTER_TYPE_AUTO	In Auto mode, bypass or bilinear is automatically selected.
E_MI_DIVP_FILTER_TYPE_BYPASS	Bypass mode.
E_MI_DIVP_FILTER_TYPE_BILINEAR	Bilinear mode.
E_MI_DIVP_FILTER_TYPE_FILTER	Number of Filter types.

- Precautions
Without scaling, the default is E_MI_DIVP_FILTER_TYPE_BYPASS filter type.
- Related data types and interfaces
[MI_DIVP_StretchBufEx](#) [#214-mi_divp_stretchbufex]

3.5. MI_DIVP_OutputPortAttr_t

- illustrate
Define the attribute parameter of the output port of the DIVP binding.
- definition

```
typedef struct MI_DIVP_OutputPortAttr_s  
{  
  
    MI_U32    u32Width ; // output width  
  
    MI_U32    u32Height ; // output height  
  
    MI_SYS_PixelFormat_e  ePixelFormat ;  
  
}
```

```
MI_SYS_CompressMode_e  eCompMode ; // compress  mode

} MI_DIVP_OutputPortAttr_t ;
```

• member

member name	describe
u32Width	The width of the output screen of the DIVP channel
u32Height	The height of the output picture of the DIVP channel
ePixelFormat	The pixel format of the output picture of the DIVP channel
eCompMode	DIVP channel output image compression format, DIVP channel can only output images in uncompressed format

• Precautions

Different series of chip DIVP output attributes

Chip Family\Output Properties	Output Pixel format
MSR930	YUV422/YUV420(NV12)/ARGB8888/ABGR8888/ARGB1555/MST420
Pretzel	YUV422
	YUV420(NV12)
	ARGB8888/ABGR8888
	RGB565
Macaron	YUV422
	YUV420(NV12)

	ARGB8888/ABGR8888
	RGB565
Taiyaki/Takoyaki	YUV422
	YUV420(NV12)
	ARGB8888/ABGR8888
	RGB565
Pudding	YUV422
	YUV420(NV12)
	ARGB8888/ABGR8888
	RGB565
Ispahan	YUV422
	YUV420(NV12)
	ARGB8888/ABGR8888
	RGB565
Ikayaki	YUV422
	YUV420(NV12)
	ARGB8888/ABGR8888
	RGB565

3.6. MI_DIVP_ChnAttr_t

- illustrate

Define the attribute parameters of the DIVP channel.

- definition

```
typedef struct MI_DIVP_ChnAttr_s
{
    MI_U32  u32MaxWidth ; // support max input width
    MI_U32  u32MaxHeight ; // support max input height
    MI_DIVP_TnrLevel_e  eTnrLevel ; // TNR level
    MI_DIVP_DiType_e  DiType ; // DI type
    MI_SYS_Rotate_e  eRotateType ; // rotate angle
    MI_SYS_WindowRect_t  stCropRect ; // crop information
    MI_BOOL  bHorMirror ; // horizontal mirror
    MI_BOOL  bVerMirror ; // vertical mirror
} MI_DIVP_ChnAttr_t ;
```

- member

member name	describe
u32MaxWidth	The maximum width of the input supported by the DIVP channel
u32MaxHeight	The maximum height of the input supported by the DIVP channel
eTnrLevel	Level of TNR on DIVP channel
eDiType	Type of DI on DIVP channel
eRotateType	The rotation angle of the picture on the DIVP channel
stCropRect	Crop information on DIVP channel
bHorMirror	Horizontal flip on DIVP channel
bVerMirror	Vertical flip on DIVP channel

• Precautions

Chip Family\Input Properties	Input Pixel format	Input align
MSR930	YUV422/YUV420(NV12)/ARGB8888/ABGR8888/ARGB1555/Tile Mode	32
Pretzel	YUV422	32
	YUV420(NV12)	16
	ARGB8888/ABGR8888	64
	RGB565	32
Macaron	YUV422	32
	YUV420(NV12)	16

	ARGB8888/ABGR8888	64
	RGB565	32
Taiyaki/Takoyaki	YUV422	32
	YUV420(NV12)	16
	ARGB8888/ABGR8888	64
	RGB565	32
Pudding	YUV422	32
	YUV420(NV12)	16
	ARGB8888/ABGR8888	64
	RGB565	32
Ispahan	YUV422	32
	YUV420(NV12)	16
	ARGB8888/ABGR8888	64
	RGB565	32
Ikayaki	YUV422	32
	YUV420(NV12)	16
	ARGB8888/ABGR8888	64

3.7. MI_DIVP_DirectBuf_t

- illustrate

Define memory information for image processing

- definition

```
typedef struct MI_DIVP_DirectBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat ; // YUV420SP or
    ARGB8888 only

    MI_U32 u32Width ;

    MI_U32 u32Height ;

    MI_U32 u32Stride [ 3 ];

    MI_PHY phyAddr [ 3 ];

} MI_DIVP_DirectBuf_t ;
```

- member

member name	describe
ePixelFormat	the pixel format of the image
u32Width	The width of the image screen
u32Height	The height of the image screen
u32Stride	The number of bytes occupied by each line of the image
phyAddr	The starting physical address of the buffer

- Precautions
 - Only supports YUV420SP or ARGB8888 image pixel format
 - u32Stride cannot be smaller than 64

- Related data types and interfaces

[MI_DIVP_StretchBuf](#) [#211-mi_divp_stretchbuf]

3.8. MI_DIVP_InitParam_t

- illustrate
DIVP device initialization parameters
- definition

```
typedef struct MI_DIVP_InitParam_s
{
    MI_U32  u32DevId ;

    MI_U8  * u8Data ;
} MI_DIVP_InitParam_t ;
```

- member

member name	describe
u32DevId	Device ID
u8Data	data pointer buffer

- Related data types and interfaces
[MI_DIVP_InitDev](#) [#212-mi_divp_initdev]

4. DIVP error code

Table 4-1 DIVP API return value

error code	macro definition	describe
0x0	MI_SUCCESS	succeeded
0xa00c2002	MI_DIVP_ERR_INVALID_CHNID	invalid channel id
0xa00c2003	MI_DIVP_ERR_INVALID_PARAM	Invalid parameter passed in
0xa00c2006	MI_DIVP_ERR_NULL_PTR	null pointer exception
0xa00c201c	MI_DIVP_ERR_FAILED	DIVP operation failed
0xa00c2005	MI_DIVP_ERR_NO_RESOUC	no resources available
0xa00c201c	MI_DIVP_ERR_NO_CONTENT	Nothing is displayed in the channel

5. Introduction to PROCFS

5.1. cat

- debug info

```
# cat /proc/mi_modules/mi_divp/mi_divp0
```

```
-----Common info for mi_divp-----
ChnNum  EnChnNum  PassNum  InPortNum  OutPortNum  CollectSize
    16         1         1         1         1         0

-----CMDQ kickoff counter-----
DevId    current_buf_size  Peak_buf_size
    0         0         0

each dev buf info:
      offset              length          used_flag          task_name

-----Common info formi_divp only dump enabled chn-----
ChnId  PassNum  EnInPNum  EnOutPNum  MMAHeapName
    0         0         1         1      (null)
ChnId    current_buf_size  Peak_buf_size  user_pid
    0         0         4c000      609

each chn buf info:
      offset              length          used_flag          task_name

-----Input port common info for mi_divp  only dump enabled port-----
ChnId  PassId  PortId  user_buf_quota  UserInjectQ_cnt  BindInQ_cnt  TotalPendingBuf_size  usrLockedInjectCnt
    0         0         0         4         0         0         0         0
ChnId  PassId  PortId  newPulseQ_cnt  nextTodoPulseQ_cnt  curWorkingQ_cnt  workingTask_cnt  lazyRewindTask_cnt
    0         0         0         0         0         0         0         0

ChnId  PassId  PortId  Enable  bind_module_id  bind_module_name  bind_ChnId  bind_PortId  bind_Type  bind_Param  enable

ChnId  PassId  PortId  SrcFrmrate  DstFrmrate  GetFrame/Ms  FPS  FinishCnt  RewindCnt
    0         0         0      0/ 0      0/ 0      0/ 1  0.00      11612      0

-----Output port common info for mi_divp  only for enabled port-----
ChnId  PassId  PortId  usrDepth  BufCntQuota  usrLockedCnt  totalOutPortInUsed  DrvBkRefFifoQ_cnt  DrvBkRefFifoQ_size
    0         0         0         5         5         0         0         0         0
ChnId  PassId  PortId  UserGetFifoQ_cnt  UserGetFifoQ_size  UserGetFifoQ_seqnum  UserGetFifoQ_discardnum
    0         0         0         0         0         11612         0
ChnId  PassId  PortId  workingTask_cnt  finishedTask_cnt
    0         0         0         0         0
ChnId  PassId  PortId  GetFrame/Ms  FPS  FinishCnt  RewindCnt  GetTotalCnt  GetOkCnt
    0         0         0      10/1100  9.09      11612      0      11612      11612

-----BindPeerInputPortList-----
ChnId  PassId  PortId  Enable  bind_module_id  bind_module_name  bind_ChnId  bind_PortId  bind_Type  bind_Param  enable

----- Start dump DEV -----
ISRNum
11612
----- End dump DEV -----

----- start dump CHN info -----
ChnId  Status  AttrChg  FieldType0  FieldType1
    0         2         0         0         0
   16         1         0         0         0

      ChnId  CropX  CropY  CropW  CropH  bHMirror  bVMirror  eDiType  eRotateType  eTnrLevel
ChnAttrPre    0     0     0    352   288      0      0      0      0      0
ChnAttrOrg    0     0     0    352   288      0      0      0      0      0
ChnAttrPre   16     0     0     0     0      0      0      0      0      0
ChnAttrOrg   16     0     0     0     0      0      0      0      0      0

----- end dump CHN info -----

----- start dump OUTPUT PORT info -----
ChnId  OutputChg  CompMode  PreWidth  PreHeight  Pixel  Stride  phyaddr  FinishTaskCnt  fps
    0         0         0      352     288     11     352  3030a000      11612     10
   16         0         0     0     0      0      0      0         0         0

----- end dump OUTPUT PORT info -----
```

- Analysis of debug information

Record the current usage of a DIVP device, as well as device attributes, layer attributes, and inputport/outputport attributes, which can be dynamically obtained for debugging and testing.

- Parameter Description

parameter		describe
channelinfo	ChnId	0~16 16 Capture Channel not open to upper layer

Status	0: INITED 1: CREATED 2: STARTED 3: STOPED 4: DISTROYED
AttrChg	0: not changed 1: changed
FieldType0, FieldType1 (last two fields's type)	0: NONE //no field. 1: TOP //Top field only. 2: BOTTOM //Bottom field only. 3: BOTH //Both fields. 4: NUM
ChnAttrPre	The last set property of the channel
ChnAttrOrg	The original set property of the channel
CropX, CropY, CropW, CropH	The crop position set by the upper layer
bHMirror bVMirror	Horizontal flip enable Vertical flip enable 0:disable 1:enable
eDiType	0: OFF,//off 1: 2D,///2.5D DI 2: 3D,///3D DI 3: NUM
eRotateType	0: NONE, //Rotate 0 degrees 1: 90, //Rotate 90 degrees 2: 180, //Rotate 180 degrees 3: 270, //Rotate 270 degrees 4: NUM
eTnrLevel	0: OFF, 1: LOW, 2: MIDDLE, 3: HIGH, 4: NUM

inputport info	ChnId	Channel ID 0~16
	InputChg	Whether the input port property has changed
	bIPChg	0: no change between interlaced and progressive 1: switch from interlaced to progressive, or progressive to interlaced
	PreWidth	input port width
	PreHeight	input port height
	Pixel	E_MI_SYS_PIXEL_FRAME_YUV422_YUYV = 0, E_MI_SYS_PIXEL_FRAME_ARGB8888, E_MI_SYS_PIXEL_FRAME_ABGR8888, E_MI_SYS_PIXEL_FRAME_RGB565, E_MI_SYS_PIXEL_FRAME_ARGB1555, E_MI_SYS_PIXEL_FRAME_I2, E_MI_SYS_PIXEL_FRAME_I4, E_MI_SYS_PIXEL_FRAME_I8, E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_4 E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_4 E_MI_SYS_PIXEL_FRAME_YUV_MST_420, //vdec mstar private video format E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE1_H: E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE2_H: E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE3_H: E_MI_SYS_PIXEL_FRAME_FORMAT_MAX
	Stride	Input's stride
	2Pmode	b2P_Enable value, 1 means enable, 0 means disable
	EnqueueTask	The number of tasks in the Enqueue
	GetInTaskCnt	The number of tasks of OnPreProcessInputTasI
Outputport Info	ChnId	Channel ID 0~16
	OutputChg	Whether the Output property has changed 0: not change

		1: changed
	Pixel	Same as input pixel
	CompMode	0 //no compress 1 //compress unit is 256 bytes as a segment 2 //compress unit is the whole line 3 //compress unit is the whole frame 4 //number
	PreWidth PreHeight	The last set output port width and height
	Stride	Output stride
	phyaddr	The physical address of the output buffer
	FinishTaskCnt	Number of Finish tasks
	fps	Frame rate of the output port
	ChnNum	total number of channels
	Pixel	Same as input pixel
Common Info	PassNum	Total number of Passes
	InPortNum	Number of Input ports
	OutPortNum	Number of Output ports
	CollectSize	The number of attached pstAllocators
	current_buf_size	The currently requested memory size
	Peak_buf_size	peak memory usage
	user_pid	process id
	user_buf_quota	The maximum number of buffers that a User can take
	UsrInjectQ_cnt	Number of User inject buffers

BindInQ_cnt	The number of buffers that the front end gives to the input port enqueue
TotalPendingBuf_size	The total buffer size occupied by each task in the current input working queue
usrLockedInjectCnt	The number of buffers obtained by the user
newPulseQ_cnt	Number of buffers for new_pulse_fifo_inputqueue
nextTodoPulseQ_cnt	The number of buffers of next_todo_pulse_inputqueue
curWorkingQ_cnt	The number of buffers of cur_working_input_queue
workingTask_cnt	The number of tasks in input_working_tasklist
lazyRewindTask_cnt	The number of tasks that need to be retried
bind_module_id	Binding module id
bind_module_name	Binding module name
bind_ChnlId	The channel id of the binding module
bind_PortId	port id of the binding module
bind_Type	绑定方式： E_MI_SYS_BIND_TYPE_FRAME_BASE = 0x00000001, E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY = 0x00000002, E_MI_SYS_BIND_TYPE_REALTIME = 0x00000000 E_MI_SYS_BIND_TYPE_HW_AUTOSYNC = 0x00000008, E_MI_SYS_BIND_TYPE_HW_RING = 0x00000010
bind_Param	Binding parameters
SrcFrmrate	source frame rate
DstFrmrate	destination frame rate

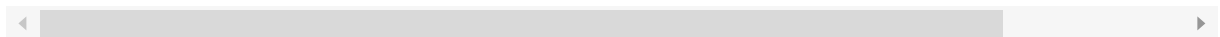
GetFrameRate	destination frame rate
GetFrame/Ms	Actual frames/time taken
FPS	actual frame rate
RewindCnt	The number of tasks that need to be retried
usrDepth	User can get the maximum number of output port buffers
BufCntQuota	Output The maximum number of buffers that can be applied for
usrLockedCnt	The number of buffers actually obtained by the User
totalOutPortInUsed	The number of buffers actually applied by Output
DrvBkRefFifoQ_cnt	for special driver which need to refer back pre-processed buffer
DrvBkRefFifoQ_size	The total number of buffer sizes occupied by stDrvBkRefFifoQueue
UsrGetFifoQ_cnt	The number of buffers from Duplicate to output port
UsrGetFifoQ_size	The total size of the buffer from Duplicate to the output port
UsrGetFifoQ_seqnum	The total number of buffers obtained by output
UsrGetFifoQ_discardnum	Output discards the original number of buffers according to the request for a new buffer
finishedTask_cnt	Number of output finish tasks
GetTotalCnt	Output gets the total number of buffers
GetOkCnt	Count the number of frames whose index is E_MI_SYS_FRC_OBTAIN

ISRNum	Total number of interrupts received
--------	-------------------------------------

5.2. echo

Function	Print the PTS of each frame received. If the front end is VDEC, the FrameID sent by t be printed to debug the frame loss problem. Under normal circumstances, the value and PTS are incremented.
Order	echo checkframeid [ChnID] [Status] > /proc/mi_modules/mi_divp/mi_divp0
Parameter Description	[ChnID] Channel number [0~32] [Status] ON Start printing OFF Stop printing
Example	echo checkframeid 0 ON/OFF > /proc/mi_modules/mi_divp/mi_divp0 [MI DIVP PROCFS]:ChnID 0, receive buffer ID = 63 , H = 0, L = 196615, PTS = 439560([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 64 , H = 0, L = 196616, PTS = 439890([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 65 , H = 0, L = 196615, PTS = 440220([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 66 , H = 0, L = 196615, PTS = 440550([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 67 , H = 0, L = 196615, PTS = 440880([MI DIVP PROCFS] :ChnID 0, receive buffer ID = 68 , H = 0, L = 196615, PTS = 441210 [MI DIVP PROCFS]:ChnID 0, receive buffer ID = 69 , H = 0, L = 196614, PTS = 441540([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 70 , H = 0, L = 196615, PTS = 441870(receive buffer ID: The ID of the buffer received by each channel. H/L: FrameID sent by VDEC, H high 32 bits, L low 32 bits, if the front end is not VDEC such printing. PTS: FrameBuffer's PTS.

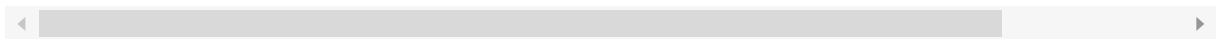
Function	Count the number of GetBuffers within 1 second, as well as the average, maximum, values of the interval between each two inputbuffers obtained. Debug frame loss.
Order	echo statintervaltime [ChnID] ON/OFF > /proc/mi_modules/mi_divp/mi_divp0
Parameter Description	[ChnID] Channel number [0~32] [Status] ON Start statistics and print OFF Stop statistics and print
Example	echo statintervaltime 0 ON > /proc/mi_modules/mi_divp/mi_divp0 [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 50393 us,MAX 59601 us,MIN 41401 us,57534 us,42415 us, 42896 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 50247 us,MAX 63004 us,MIN 36896 us,37120 us,37130 us [MI DIVP PROCFS]:ChnIID 0, 1s FrameCnt:18,Average time: 51049us,MAX 70431us,59763us,59736us,MIN 38873us,40109us,40283us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:20,Average time: 48537us, MAX 67168 us, MIN 18513 us, 31242 us, 32681 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 49962 us,MAX 61416 us,MIN 38564 us, 38736 us, 38762 us FrameCnt: The number of GetBuffers in 1 second Average time: The average time between getting two buffers MAX: The three values with the largest interval time. MIN: The three values with the smallest interval time



Function	DIVP no longer processes a certain channel to save bandwidth and verify bandwidth issues
Order	echo stoponechannel [ChnID] [Status] > /proc/mi_modules/mi_divp/mi_divp0
Parameter Description	[ChnID] Channel number [0~32] [Status] ON Stop processing the set channel OFF continue processing the set channel
Example	echo stoponechannel 0 ON > /proc/mi_modules/mi_divp/mi_divp0 The screen of channel 0 will stop



Function	Dump an input frame.
Order	echo dumpframe [ChnID] [frameStorePath] > /proc/mi_modules/mi_divp/mi_divp0
Parameter Description	[ChnID] Channel number [0~16] [frameStorePath] Specify the location to save the input frame.
Example	#echo dumpframe 0 /customer > /proc/mi_modules/mi_divp/mi_divp0 Dump An input frame is saved in the path /customer. Default input frame name such as: dump_divp_chn0_frame_w_352_h_288.yuv



Function	Open the debug information of mi divp.
Order	echo debug 0/1 > /proc/mi_modules/mi_divp/mi_divp0
Parameter Description	[0/1] 0 means off, 1 means on.
Example	#echo debug 1 > /proc/mi_modules/mi_divp/mi_divp0 printk in _MI_SYS_IMPL_Common_WriteProc divp debug:on #echo debug 0 > /proc/mi_modules/mi_divp/mi_divp0 printk in _MI_SYS_IMPL_Common_WriteProc divp debug:off

